

学号：22920202200773

姓名：孟媛

实验 8 多项式拟合

一、实验目的

使用多项式拟合进行函数逼近。

二、数值计算原理

多项式拟合即对于一组数据 (x_i, y_i) ， $i=1, 2, \Lambda, n$ ，从函数类

$\Phi = \{\varphi_0(x), \varphi_1(x), \Lambda, \varphi_m(x)\}$ 中找到一个函数 $p(x)$ ，使误差的 2-范数平方：

$$\|\delta\|_2^2 = \sum_{i=0}^n \delta_i^2 = \sum_{i=0}^n (p(x_i) - y_i)^2$$

达到最小。

当 $\varphi_0(x), \varphi_1(x), \Lambda, \varphi_m(x)$ 是 Φ 的一组线性无关的基函数时， $p(x)$ 为 $\{\varphi_i(x)\}$ 的线性组合，

即： $p(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \Lambda + a_m\varphi_m(x) (m < n)$

将其带入 $\|\delta\|_2^2 = \sum_{i=0}^n \delta_i^2 = \sum_{i=0}^n (p(x_i) - y_i)^2$ ，并通过导数求解最小值，有：

$$\sum_{j=0}^n (\varphi_k, \varphi_j) a_j = d_k \quad (k = 0, 1, 2, \Lambda, m)$$

矩阵形式为：

$$\begin{pmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \Lambda & (\varphi_0, \varphi_n) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \Lambda & (\varphi_1, \varphi_n) \\ \text{M} & \text{M} & \text{M} & \text{M} \\ (\varphi_n, \varphi_0) & (\varphi_n, \varphi_1) & \Lambda & (\varphi_n, \varphi_n) \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \text{M} \\ a_n \end{pmatrix} = \begin{pmatrix} (f, \varphi_0) \\ (f, \varphi_1) \\ \text{M} \\ (f, \varphi_n) \end{pmatrix}$$

这就是关于系数的线性方程组，也称法方程，求解法方程，就可得到结果。

特别的，当 $\Phi = \{\varphi_0, \varphi_1, \Lambda, \varphi_m\} = \{1, x, x^2, \Lambda, x^m\}$ 时，有：

$$\begin{pmatrix} \sum 1 & \sum x_i & \Lambda & \sum x_i^m \\ \sum x_i & \sum x_i^2 & \Lambda & \sum x_i^{m+1} \\ M & M & M & M \\ \sum x_i^m & \sum x_i^{m+1} & \Lambda & \sum x_i^{2m} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ M \\ a_n \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ M \\ \sum x_i^m y_i \end{pmatrix}$$

三、源程序介绍

函数接受三个参数：数据向量及其对应的函数值和多项式的次数，返回多项式的升幂排列系数。

函数计算过程：

- (1) 计算法方程组系数矩阵
- (2) 求解法方程，得到多项式的系数

```

1  #多项式拟合
2  import numpy
3  import math
4  def func(x, y, mi):
5      a = numpy.zeros((mi + 1, mi + 1))
6      b = numpy.zeros((mi + 1, 1))
7      for i in range(mi + 1):
8          for j in range(mi + 1):
9              a[i, j] = numpy.sum(x ** (i + j))
10             b[i, 0] = numpy.sum(x ** i * y)
11     c = numpy.linalg.inv(a)
12     c = numpy.dot(c, b)
13     return numpy.transpose(c)
14
15 x = numpy.array([2,4,6,8])
16 y = numpy.array([2,11,28,40])
17 print("自编函数升序排列系数为:", func(x, y, 1))
18 print("自带函数polyfit降幂排列系数为:", numpy.polyfit(x, y, 1))

```

四、程序执行情况

4.1 书上给的例子：

```

x = numpy.array([2,4,6,8])
y = numpy.array([2,11,28,40])

```

```

/myy\01\数值计算第二版\例题22920202200\73\题解\例题
自编函数升序排列系数为: [[-12.5    6.55]]
自带函数polyfit降幂排列系数为: [ 6.55 -12.5 ]

```

4.2 书上未给的例子

```
x = numpy.array([2,8,6,8])  
y = numpy.array([2,64,28,40])
```

/myyのAI/数值计算第二次实验22920202200773孟媛/实验八/2292020

自编函数升序排列系数为: $\begin{bmatrix} -16.5 & 8.33333333 \end{bmatrix}$

自带函数polyfit降幂排列系数为: $\begin{bmatrix} 8.33333333 & -16.5 \end{bmatrix}$

五、结果分析

通过对比，可以看到多项式拟合的结果和 `polyfit` 工具函数的结果基本一致，精度非常高，可以满足一般的函数逼近要求。

六、实验体会

多项式拟合思路简单，编程难度不大，代码短小，精度特别高。并且我对函数逼近问题、多项式拟合的方法都有了进一步的认识。